Group Chat Sockets App

Using Python to create a chat application in the Terminal console. It uses sockets to facilitate communication between a server (that handles group communications) and directly from client to client (in private messages).

To setup the server:
python3 ChatApp.py -s <port_number>

To setup a client (will need at least 2):
Python3 ChatApp.py -c <name> <server_ip> <server_port> <client_port>

Basic Commands:

1) send <name> <msg>	—	sends msg to any connected clients with given name
2) dereg <name>	—	deregisters client from the server's log of clients
3) create_group <group_name>	—	prompts server to create a new group with the given group name
4) list_groups	—	requests list of currently available groups from server
5) join_group <group_name>	—	asks server to join given group

Once a user runs join_group, they may use the following commands…

Group Chat Commands:

1) send_group <msg>	—	sends msg to server, server distributes msg to all clients in group
2) list_members	—	requests list of members of current group from server
3) leave_group	—	exit group (can use basic commands again)
4) dereg <name>	—	see dereg in basic commands

```
import socket
from socket import *
from tabulate import tabulate
import time
import threading
from signal import signal, SIGINT, default_int_handler
import sys
import os

user_input = sys.argv
ip = gethostbyname(gethostname())
name, port, mode = "", "", ""
in_group, group_str = "", ""
success, sent = False, False
client_acks = []
msgs_while_away = []
clients = []
```

```
groups = {}

# Expected terminal input...
# Server: python3 ChatApp.py -s <port>
# Client: python3 ChatApp.py -c <name> <server_ip> <server_port> <client_port>

# Packet structure:
# Line 1  :  header (register, chat, de-register, etc.)
# Line 2  :  client information (name, ip, port)
# Line 3  :  msg
# Line 4+ :  possible additional information


def dereg_client(send_sock, bad_client):
    global clients
    for nested_list in clients:
        if (bad_client in nested_list) and not (nested_list[3] == '0'):
            nested_list[3] = '0'
            broadcast = threading.Thread(target=broadcast_client_table,
args=(send_sock,))
            broadcast.start()
            print(">>> [Client {} went offline.]".format(bad_client))
            print("\n" + tabulate(clients, headers=["NAME", "IP", "PORT",
"ONLINE-STATUS"]) + "\n")


def get_info_for_name(name):
    send_info = ('', 0)
    for nested_list in clients:
        for item in nested_list:
            if (item == name):  # and (nested_list[3] == '1'):
                send_info = (nested_list[1], int(nested_list[2]))
    return send_info


def name_in_clients(name):
    global clients
    return any(name in nested_list for nested_list in clients)


def nested_list_from_str(list_str):
    return_list = []
    lists = [l.strip('[]') for l in list_str.split("], [")]
    for row in lists:
        return_list.append([l.strip("'") for l in row.split(", ")])
    return return_list


def print_above(msg):
```

```
    # Use ANSI escape codes to print above the input line
    # NOTE: for msg's longer than the length of terminal, need to call
print_above() on portions

    cursor_down_one_line = "\033[1B"
    cursor_up_one_line = "\033[1A"
    scroll = "\033[1S"
    insert_line_above = "\033[1L"
    cursor_save_horiz_loc = "\033[s"
    cursor_return = "\033[u"
    print(cursor_save_horiz_loc + scroll + cursor_up_one_line +
insert_line_above + msg +
          cursor_return, end="", flush=True)


def valid_boot():
    global name, port, mode
    if len(user_input) < 2:
        raise Exception("Specify server ( -s ) or client ( -c ).")
    mode = user_input[1]
    if mode == '-s':
        if len(user_input) < 3:
            raise Exception("Specify port number in range 1024-65535.")
        elif not valid_port(user_input[2]):
            raise Exception("Invalid port number out of range 1024-65535.")
        name, port = "", user_input[2]
    elif mode == '-c':
        if len(user_input) < 6:
            raise Exception("Insufficient arguments. Client requires: <name>
<server_ip> <server_port> <client_port>")
        elif ('(' in user_input[2]) or (',' in user_input[2]):
            raise Exception("Please do not include commas in your name field.")
        elif not valid_ip(user_input[3]):
            raise Exception("Invalid IP. Use IPv4 format x.x.x.x where 0 <= x <=
255.")
        elif not valid_port(user_input[4]):
            raise Exception("Invalid server port number out of range
1024-65535.")
        elif not valid_port(user_input[5]):
            raise Exception("Invalid client port number out of range
1024-65535.")
        name, port = user_input[2], user_input[5]
    else:
        raise Exception("Invalid boot option. Specify server ( -s ) or client (
-c ).")


def valid_ip(ip):
```

```
    if ip == "localhost":
        return True
    octets = ip.split(".")
    if len(octets) != 4:
        return False
    for o in octets:
        if not(o.isnumeric() and (0 <= int(o) <= 255)):
            return False
    return True


def valid_port(port):
    if port.isnumeric() and (1023 < int(port) <= 65535):
        return True
    return False


def send_ack(sock_send, addr_dest, port_dest, msg=None):
    global name, ip, port
    ack = "ack\n" + ', '.join([name, ip, port]) + "\n" + str(msg) + "\n"
    sock_send.sendto(ack.encode(), (addr_dest, int(port_dest)))
    return 1


def send_nak(sock_send, addr_dest, port_dest, msg=None):
    global name, ip, port
    nak = "nak\n" + ', '.join([name, ip, port]) + "\n" + str(msg) + "\n"
    sock_send.sendto(nak.encode(), (addr_dest, int(port_dest)))
    return 1


def broadcast_client_table(sock_send):
    global name, ip, port, clients
    pkt = "update_clients\n" + ', '.join([name, ip, port]) + "\n" + str(clients)
+ "\n"
    for client in clients:
        if client[3] == "1":
            sock_send.sendto(pkt.encode(), (client[1], int(client[2])))
    return 1


def notify_server_of_offline_client(sock_send, addr_dest, port_dest, msg):
    global name, ip, port
    pkt = "offline_client\n" + ', '.join([name, ip, port]) + "\n" + str(msg) +
"\n"
    sock_send.sendto(pkt.encode(), (addr_dest, int(port_dest)))
    return 1
```

```
def reg_client(send_sock, info):
    global clients
    name_c, ip_c, port_c = info.split(', ')

    # NAK if name is already taken
    if name_in_clients(name_c):
        nak_reg = threading.Thread(target=send_nak, args=(send_sock, ip_c,
port_c))
        nak_reg.start()
    else:
        # Otherwise, ACK
        ack_reg = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c))
        ack_reg.start()
        table_entry = [name_c, ip_c, port_c, "1"]

        # Update client table
        if table_entry not in clients:
            if [name_c, ip_c, port_c, "0"] in clients:
                clients.remove([name_c, ip_c, port_c, "0"])
            clients.append(table_entry)
            print(">>> [Registered new client {}.]".format(name_c))
            print("\n" + tabulate(clients, headers=["NAME", "IP", "PORT",
"ONLINE-STATUS"]) + "\n")
            broadcast = threading.Thread(target=broadcast_client_table,
args=(send_sock,))
            broadcast.start()


def handle_create_group(send_sock, group_name, dest_info):
    global groups
    name_c, ip_c, port_c = dest_info.split(', ')
    msg = ""

    if group_name in groups.keys():
        msg = "create_group\n>>> [Group {} already exists.]".format(group_name)
        print(">>> [Client {} creating group {} failed, group already
exists]".format(name_c, group_name))
    else:
        groups[group_name] = []
        msg = "create_group\n>>> [Group {} created by
Server.]".format(group_name)
        print(">>> [Client {} created group {} successfully]".format(name_c,
group_name))

    ack_create_group = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, msg))
    ack_create_group.start()
```

```
def handle_join_group(send_sock, group_name, info):
    global groups
    name_c, ip_c, port_c = info.split(', ')
    msg = ""

    if group_name in groups.keys():
        groups[group_name].append(name_c)
        msg = "join_group\n>>> [Entered group {}
successfully]".format(group_name)
        print(">>> [Client {} joined group {}]".format(name_c, group_name))
    else:
        msg = "join_group\n>>> [Group {} does not exist]".format(group_name)
        print(">>> [Client {} joining group {} failed, group does not
exist]".format(name_c, group_name))

    ack_join_group = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, msg))
    ack_join_group.start()


def handle_leave_group(send_sock, group_name, info):
    global groups
    name_c, ip_c, port_c = info.split(', ')
    groups[group_name].remove(name_c)
    print(">>> [Client {} left group {}]".format(name_c, group_name))
    ack_leave_group = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, "leave_group"))
    ack_leave_group.start()


def handle_list_groups(send_sock, info):
    global groups
    name_c, ip_c, port_c = info.split(', ')
    print(">>> [Client {} requested listing groups, current
groups:]".format(name_c))
    msg = ""
    for group in groups.keys():
        print(">>> <{}>".format(group))
        msg += (group + "\n")
    ack_list_groups = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, "list_groups\n" + msg))
    ack_list_groups.start()


def handle_list_members(send_sock, group_name, info):
    global groups
    name_c, ip_c, port_c = info.split(', ')
```

```
    print(">>> [Client {} requested listing members of group
{}:]".format(name_c, group_name))
    msg = ""
    for member in groups[group_name]:
        print(">>> <{}>".format(member))
        msg += (member + "\n")
    ack_list_members = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, "list_members\n" + msg))
    ack_list_members.start()


def send_group(send_sock, addr_dest, port_dest, msg):
    global name, ip, port
    pkt = "receive_group\n" + ', '.join([name, ip, port]) + "\n" + msg + "\n"
    send_sock.sendto(pkt.encode(), (addr_dest, int(port_dest)))
    return 1


def handle_send_group(send_sock, group_name, msg, info):

    global groups, client_acks

    # Log message in server
    name_c, ip_c, port_c = info.split(', ')
    print(">>> [Client {} sent group message: {}]".format(name_c, msg))

    # Send ACK to original sending client
    ack_send_group = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, "send_group"))
    ack_send_group.start()

    # Make a thread for each client that the group message will be sent to
    to_thread = []
    for client in groups[group_name]:
        client_info = get_info_for_name(client)
        if (client_info != ('', 0)) and (name_c != client):
            to_thread.append(threading.Thread(target=send_group,
                                              args=(send_sock, client_info[0],
                                                    client_info[1],
"{}\n".format(name_c) + msg)))

    # Start the threads and check which clients ACK'd at the end
    client_acks = []
    for thread in to_thread:
        thread.start()
    time.sleep(0.5)
    to_remove = []

    # Remove clients that didn't ACK from group
```

```
    for client in groups[group_name]:
        if (client not in client_acks) and (client != name_c):
            print(">>> [Client {} not responsive, removed from group
{}]".format(client, group_name))
            to_remove.append(client)
    for client in to_remove:
        groups[group_name].remove(client)




def try_five_times(send_sock, pkt):
    global success, group_str
    success = False
    for attempts in range(5):
        if success:
            return True
        # print(">>> {}[Attempt #{}...]".format(group_str, str(attempts+1)))
        send_sock.sendto(pkt.encode(), (user_input[3], int(user_input[4])))
        time.sleep(0.5)
    print(">>> {}[Server not responding]\n>>> {}[Exiting]".format(group_str,
group_str))
    os._exit(1)
    return False


def reg_request(send_sock):

    global name, ip, port

    # Try 5 times to send register request to server
    pkt = "reg\n" + ', '.join([name, ip, port]) + "\n\n"
    try_five_times(send_sock, pkt)


def send(send_sock, chat_input):

    global name, ip, port, sent

    if len(chat_input) < 3:
        print(">>> [Proper format: send <dest_name> <msg>]")
        return
    dest_name = chat_input[1]
    send_info = get_info_for_name(dest_name)
    if send_info == ('', 0):
        print(">>> [Specified dest_name not in list of clients.]")
        return
    msg = ' '.join(chat_input[2:])
    pkt = "send\n" + ', '.join([name, ip, port]) + "\n" + msg + "\n"
```

```
    # Attempt to send msg
    send_sock.sendto(pkt.encode(), send_info)
    time.sleep(0.5)
    if sent:
        print(">>> [Message received by {}]".format(dest_name))
    else:
        print(">>> [No ACK from {}, message not delivered]".format(dest_name))

        # Notify server that client is unresponsive
        notify_server_of_offline_client(send_sock, user_input[3], user_input[4],
dest_name)
    sent = False


def dereg_request(send_sock, chat_input):
    global name, ip, port, group_str

    # Verify proper input
    if (len(chat_input) != 2) or (chat_input[1] != name):
        print(">>> {}[Proper format: dereg {}]".format(group_str, name))
        return

    # Try 5 times to send dereg request to server
    pkt = "dereg\n" + ', '.join([name, ip, port]) + "\n" + chat_input[1] + "\n"
    try_five_times(send_sock, pkt)


def create_group_request(send_sock, chat_input):

    global name, ip, port

    # Verify proper input
    if len(chat_input) != 2:
        print(">>> [Proper format: create_group <group_name>]")
        return
    group_name = chat_input[1]

    # Try 5 times to send create_group request to server
    pkt = "create_group\n" + ', '.join([name, ip, port]) + "\n" + group_name +
"\n"
    try_five_times(send_sock, pkt)


def list_groups_request(send_sock, chat_input):

    global name, ip, port

    # Verify proper input
```

```
    if len(chat_input) != 1:
        print(">>> [Proper format: list_groups]")
        return

    # Try 5 times to send list_groups request to server
    pkt = "list_groups\n" + ', '.join([name, ip, port]) + "\n\n"
    try_five_times(send_sock, pkt)


def join_group_request(send_sock, chat_input):

    global name, ip, port

    # Verify proper input
    if len(chat_input) != 2:
        print(">>> [Proper format: join_group <group_name>]")
        return
    group_name = chat_input[1]

    # Try 5 times to send join_group request to server
    pkt = "join_group\n" + ', '.join([name, ip, port]) + "\n" + group_name +
"\n"
    try_five_times(send_sock, pkt)


def list_members_request(send_sock, chat_input):

    global name, ip, port, in_group, group_str

    # Verify proper input
    if len(chat_input) != 1:
        print(">>> {}[Proper format: list_members]".format(group_str))
        return

    # Try 5 times to send join_group request to server
    pkt = "list_members\n" + ', '.join([name, ip, port]) + "\n" + in_group +
"\n"
    try_five_times(send_sock, pkt)


def send_group_request(send_sock, chat_input):

    global name, ip, port, in_group, group_str

    # Verify proper input
    if len(chat_input) < 2:
        print(">>> {}[Proper format: send_group <msg>]".format(group_str))
        return
```

```
    # Try 5 times to send leave_group request to server
    pkt = "send_group\n" + ', '.join([name, ip, port]) + "\n" + in_group + "\n"
+ " ".join(chat_input[1:])
    try_five_times(send_sock, pkt)


def leave_group_request(send_sock, chat_input):

    global name, ip, port, in_group, group_str

    # Verify proper input
    if len(chat_input) != 1:
        print(">>> {}[Proper format: leave_group]".format(group_str))
        return

    # Try 5 times to send leave_group request to server
    pkt = "leave_group\n" + ', '.join([name, ip, port]) + "\n" + in_group + "\n"
    try_five_times(send_sock, pkt)









def client_listen(send_sock, port):

    global name, clients, in_group, success, sent, group_str, msgs_while_away

    # Open listening socket for client
    sock_c_listen = socket(AF_INET, SOCK_DGRAM)
    sock_c_listen.bind(('', int(port)))

    # Wait for ACK regarding registration
    success = False
    while not success:
        buf, addr = sock_c_listen.recvfrom(4096)
        buf_contents = buf.decode().splitlines()
        header, info, msg = buf_contents[0:3]
        if header == "ack":
            success = True
            print(">>> [Welcome, You are registered.]")
        elif header == "nak":
            print(">>> [Name already taken.]")
            print(">>> [Exiting]")
            os._exit(0)

    # Handle further packets
```

```
   while True:
       buf, addr = sock_c_listen.recvfrom(4096)
       buf_contents = buf.decode().splitlines()
       header, info, msg = buf_contents[0:3]
       if header == "update_clients":
           clients = nested_list_from_str(msg)
           print_above(">>> {}[Client table updated.]".format(group_str))
       elif header == "send":
           name_c, ip_c, port_c = info.split(', ')
           ack_send = threading.Thread(target=send_ack, args=(send_sock, ip_c,
port_c, "send"))
           ack_send.start()
           to_print = ">>> {}: {}".format(name_c, msg)
           if in_group:
               msgs_while_away.append(to_print)
           else:
               print_above(to_print)
       elif header == "receive_group":
           name_s, ip_s, port_s = info.split(', ')
           ack_send = threading.Thread(target=send_ack, args=(send_sock, ip_s,
port_s, name))
           ack_send.start()
           if in_group:
               print_above(">>> {}Group_Message {}: {}".format(group_str, msg,
"".join(buf_contents[3:])))
       elif header == "ack":
           success = True
           if msg == "send":
               sent = True
           elif msg == "dereg":
               print(">>> [You are Offline. Bye.]")
               os._exit(0)
           elif msg == "create_group":
               print(buf_contents[3])
           elif msg == "list_groups":
               print(">>> [Available group chats:]")
               for group in buf_contents[3:len(buf_contents)-1]:
                   print(">>> <{}>".format(group))
           elif msg == "join_group":
               if "successfully" in buf_contents[3]:
                   in_group = buf_contents[3].split(' ')[3]
               print(buf_contents[3])
           elif msg == "list_members":
               print(">>> {}[Members in the group {}:]".format(group_str,
in_group))
               for member in buf_contents[3:len(buf_contents)-1]:
                   print(">>> {}<{}>".format(group_str, member))
           elif msg == "send_group":
               print(">>> {}[Message received by Server.]".format(group_str))
```

```
            elif msg == "leave_group":
                print(">>> [Leave group chat {}]".format(in_group))
                in_group = ""   # No current group

                # Print private messages received while in group chat
                print(">>> [Private messages received while in group:]")
                for message in msgs_while_away:
                    print(message)
                msgs_while_away = []
            else:
                print(buf_contents)
        else:
            print_above(">>> {}[Received unknown packet.] : ".format(group_str)
+ str([header, info, msg]))


def start_as_client(ip_s, port_s):

    global name, ip, port, clients, in_group, group_str, sent
    clients.append([name, ip, port])

    # Open socket for sending messages
    sock_c = socket(AF_INET, SOCK_DGRAM)

    # Start another socket for simultaneous listening
    listen = threading.Thread(target=client_listen, args=(sock_c, int(port)))
    listen.start()

    # Send registration request to server
    reg_request(sock_c)

    # Handle command line inputs
    while True:

        # Make sure input isn't empty, check for ctrl + c
        group_str = "({}) ".format(in_group) if in_group != "" else ""
        print(">>> " + group_str, end="")
        try:
            raw_input = input()
        except Exception as e:  # For some reason only Exception works, not
KeyboardInterrupt
            print(e)
            os._exit(0)
        chat_input = raw_input.split()
        if len(chat_input) == 0:
            print(">>> {}[Invalid command]".format(group_str))
            continue
        header = chat_input[0]
```

```
        # Send message to named client : send <dest_name> <msg>
        if header == "send" and not in_group:
            send(sock_c, chat_input)
        elif header == "dereg":
            dereg_request(sock_c, chat_input)
        elif header == "create_group" and not in_group:
            create_group_request(sock_c, chat_input)
        elif header == "join_group" and not in_group:
            join_group_request(sock_c, chat_input)
        elif header == "list_groups" and not in_group:
            list_groups_request(sock_c, chat_input)
        elif header == "list_members" and in_group:
            list_members_request(sock_c, chat_input)
        elif header == "send_group" and in_group:
            send_group_request(sock_c, chat_input)
        elif header == "leave_group" and in_group:
            leave_group_request(sock_c, chat_input)
        else:
            print(">>> {}[Invalid command]".format(group_str))


def start_as_server(port):

    global clients, groups, client_acks

    # Create server listening socket
    sock_s = socket(AF_INET, SOCK_DGRAM)                # (AF_INET : IPv4 ,
SOCK_DGRAM : using UDP)
    sock_s.bind(('', int(port)))                            # ('' : defaults to
localhost)
    sock_s.setsockopt(SOL_SOCKET, SO_BROADCAST, 1)    # Allows socket to
broadcast
    print(">>> [Server Online]")

    # Open socket for sending messages
    sock_c = socket(AF_INET, SOCK_DGRAM)

    # Listen for packets
    while True:
        buf, addr_c = sock_s.recvfrom(4096)        # Listen from port (4096 :
buf size , addr_c : (IP , port) of client)
        buf_contents = buf.decode().splitlines()    # Encode msgs before sending,
decode msgs after receiving
        header, info, msg = buf_contents[0:3]
        name_c, ip_c, port_c = info.split(', ')

        # Uncomment to see all received packets, must have tabulate library
        # print(">>> [Received packet.]")
```

```
        # print("\n" + tabulate(list([[header, info, msg]]), headers=["HEADER",
"INFO", "MSG"]) + "\n")

        # Handle register requests

        # CHANGED SOCK_S's to SOCK_C's
        # Maybe move the threads to being the handle functions instead of
calling ack in handle functions as thread

        if header == "reg":
            reg_client(sock_c, info)
        elif header == "offline_client":
            dereg_client(sock_c, msg)
        elif header == "dereg":
            dereg_client(sock_c, msg)
            ack_dereg = threading.Thread(target=send_ack, args=(sock_c, ip_c,
port_c, "dereg"))
            ack_dereg.start()
        elif header == "create_group":
            handle_create_group(sock_c, msg, info)
        elif header == "list_groups":
            handle_list_groups(sock_c, info)
        elif header == "join_group":
            handle_join_group(sock_c, msg, info)
        elif header == "list_members":
            handle_list_members(sock_c, msg, info)
        elif header == "send_group":
            t = threading.Thread(target=handle_send_group, args=(sock_c, msg, "
".join(buf_contents[3:]), info))
            t.start()
        elif header == "leave_group":
            handle_leave_group(sock_c, msg, info)
        elif header == "ack":
            # This is an ack for a send_group command
            client_acks.append(msg)


if __name__ == "__main__":

    # Ensure that the terminal input is valid and has proper IPs/ports
    valid_boot()

    # Handle ctrl + c exit
    signal(SIGINT, default_int_handler)

    # Boot in selected mode
    if mode == '-s':            # Boot in server mode
        start_as_server(int(port))
    elif mode == '-c':          # Boot in client mode
```

```
        ip_s = user_input[3]
        port_s = int(user_input[4])
        start_as_client(ip_s, port_s)
```